Lessons Learned — Food Log / Weekly Overview Feature

Notes from building the weekly tracking feature in NutriPlan. Read before starting a new feature that touches `localStorage` or state derived from multiple sources.

---

1. Never call a "write/mutate" function unconditionally on page load

What happened: `main.js` called `logMeal.saveDataToStorage()` at the top level, so it ran on *every* page load/refresh — not just when the user actually logged something.

The concept: Separate your functions into two categories and never mix them:

- **Trigger functions** — run only in response to a user action (a click, a form submit). Wire these to event listeners, never call them at the top level of a script.
- **Load functions** — run once on page load to read existing state and render it (e.g. `confirmData()`).

If a "trigger" function accidentally runs during "load," you get phantom writes that corrupt state silently, because nothing about the bug is visible until you refresh a few times.

Rule of thumb: if a function's name has "save," "add," "log," "submit," or "confirm" in it, it belongs inside an event handler — not in the top-level call list of `main.js`.

---

2. Order of operations matters when state depends on other state

What happened: Reset logic (`checkWeekReset`, `checkDayReset`) needs to run *before* anything reads or renders `weeklyLog`/`mealsArray`. If rendering happens first, you render stale or pre-reset data for one frame, or reset something after it was already used.

The concept: When one function's correctness depends on another function having already run, don't rely on remembering to call them in the right order in a few different places. Bundle them into a single "load" function (like `confirmData()`) that does, in strict order:

1. Reset/expire stale state
2. Load fresh state from storage
3. Recompute derived values
4. Render

---

3. Don't track derived numbers with increment/decrement — recompute them from the source of truth

What happened: `weeklyLog` tracked today's calories/items as a running counter (`+= calories`, `items++`). Any mismatch between when an item was added vs. removed vs. re-rendered caused permanent drift — numbers went negative, or stayed at 0 after items were clearly logged, and refreshing never fixed it because there was nothing to re-derive from.

The concept: A number that can be *calculated* from other data (like "today's total calories" from `mealsArray` + `productsArray`) should never be stored and incremented independently. Two copies of the same fact will eventually disagree.

The fix pattern — "snapshot sync":

```
function syncTodayIntoWeeklyLog(){
  // Always rebuild from the real arrays — never adjust a
  let todayCalories = 0;
  let todayItems = 0;
  for (const meal of mealsArray) { todayCalories += meal.
  for (const product of productsArray) { todayCalories +=
  weeklyLog[todayKey] = { calories: todayCalories, items:
}
```

Call this *after* any mutation to `mealsArray`/`productsArray` (add, delete, clear-all) and once on every page load. It self-heals — even if a previous bug corrupted the stored numbers, the very next sync overwrites them with the truth.

General rule: if you ever find yourself writing `someTotal += x` and `someTotal -= x` in different places, ask whether `someTotal` could instead be computed fresh from the underlying list each time it's needed.

---

4. Guard against operating on an index/item that no longer exists

What happened: `deleteProduct(index, itemType)` assumed `productsArray[index]` always existed. A stale click, double-click, or re-render with a shifted index could call it with an invalid index and silently corrupt calorie totals.

The concept: Any function that receives an index or ID from the DOM (not from your own live array) should check the item still exists before acting on it:

```
if (!productsArray[index]) return;
```

This costs one line and prevents an entire class of "how did this go

negative" bugs.

---

5. Watch for small typos in `localStorage` keys

What happened: One line used `localStorage.setItem("mealsArray ", ...)` — a trailing space in the key. It silently created a second, separate storage entry instead of updating the real one, so deletes never actually persisted for meals.

The concept: `localStorage` keys are just strings — there's no autocomplete or type-checking to catch a typo. When a value "isn't updating" but no error is thrown, check the exact key string first, especially after copy-pasting a line and editing it.

---

6. Reset logic needs two independent clocks, not one

What happened: Daily data (today's log) and weekly data (the 7-day overview) needed to expire on *different* schedules — daily resets every calendar day, weekly resets every Monday. Using a single "last seen date" would have reset both together.

The concept: When two pieces of state have different lifecycles, store and check their expiry independently:

- `lastActiveDate` → compared to today's date key, resets daily data
- `weekStartDate` → compared to this week's Monday, resets weekly data

Don't try to derive one from the other or share a single timestamp for

both.